

Cassandra vs CockroachDB

A reference for students arriving from Cassandra. Both are distributed, shared-nothing, horizontally scalable, multi-datacenter databases with no single point of failure. From there they diverge on almost every design axis, and most of the divergence follows from one choice: Cassandra places data by **hash**, CockroachDB by **sort order**.

At a Glance

	Cassandra	CockroachDB
Data model	Wide-column: partition key + clustering columns, denormalised per query	Relational: tables, schemas, foreign keys, normalised as you like
Query language	CQL — SQL-like; no joins, subqueries, or cross-partition aggregation	PostgreSQL-dialect SQL: joins, CTEs, window functions
Wire protocol	Its own; Cassandra drivers only	PostgreSQL — any PG driver, ORM, or tool
Consistency	Tunable per query; eventual by default; last-write-wins	SERIALIZABLE, always; strongly consistent
Transactions	Single partition only (Paxos lightweight transactions)	ACID across any rows, tables, or ranges
Placement	Hash of the partition key onto a token ring	Sorted key space cut into dynamic ranges
Coordination	Leaderless; every replica is a peer	Raft leader / leaseholder per range
Write path	Any replica accepts; acknowledged at the consistency level you chose	Leaseholder proposes; acknowledged at quorum, always
Reads	Any replica; digest comparison at higher levels	Leaseholder only, or a follower at a past timestamp
Secondary indexes	Local per node; queries fan out; generally discouraged	Global, consistent, first-class
Schema changes	Online, eventually propagated	Online, transactional
Hot key handling	Redesign the partition key	Load-based range splits and lease moves; a hot single row still needs a key redesign
Multi-region	<code>NetworkTopologyStrategy</code> per keyspace; application picks consistency per query	Declarative per database and table: <code>REGIONAL BY ROW</code> , <code>GLOBAL</code> , survival goals
Storage engine	LSM (SSTables); compaction strategy is your choice	LSM (Pebble); managed
Language / runtime	Java; JVM tuning is part of operating it	Go; one static binary

	Cassandra	CockroachDB
Licence	Apache 2.0	Source-available (BSL, converting to Apache 2.0 three years after each release for v23.2); enterprise features need a key

Partitions vs Ranges

	Cassandra partition	CockroachDB range
Boundaries	Fixed by the hash function; a partition never splits	Dynamic: splits on size (512 MiB default) or load; merges when small
Where a key lands	Wherever its hash says — uniform by construction	Next to its neighbours in key order — locality by construction
Sequential keys	Spread automatically	Hotspot on the right edge unless you use UUIDs or hash-sharding
Range scans across the unit	Not efficient — no global order	Efficient — a prefix scan is one contiguous read
Size control	None	<code>range_min_bytes</code> / <code>range_max_bytes</code> in the zone config, at any level
Manual boundaries	None	<code>ALTER TABLE ... SPLIT AT</code> , released with <code>UNSPLIT ALL</code>

What Cassandra gets for free that CockroachDB makes you design for: even write distribution. Hashing every key is exactly what a hash-sharded primary key does in CockroachDB — Playbook pattern #1 is Cassandra's default behaviour, opted into per table. Cassandra's clustering columns (sorted within a partition) are the analogue of the composite key suffix in pattern #3.

What CockroachDB gets that Cassandra gives up: ordered scans, joins, multi-row atomicity, and the ability to split a hot range by load and move its lease. A hot Cassandra partition can only be fixed by changing the key; a hot CockroachDB range can often be split — though a single hot *row* cannot be, which is why the sharded counter pattern exists in both worlds.

Keyspaces vs Databases

Physically, a CockroachDB cluster is **one** sorted key space: every table lives under a prefix built from its ID (`/Table/108/1/...`), and ranges are cut across that space regardless of which database a table belongs to. Logically, the Cassandra keyspace maps to a CockroachDB **database** — a namespace for tables, grants, and settings, and the level at which multi-region configuration is set.

Cassandra sets replication strategy and factor **per keyspace**. CockroachDB's equivalent is the **zone configuration**, which attaches at any level: cluster default, database, table, index, or partition.

```
ALTER DATABASE bank CONFIGURE ZONE USING num_replicas = 5; -- the keyspace-level setting
ALTER TABLE bank.audit CONFIGURE ZONE USING num_replicas = 3; -- overridden for one table
```

What does **not** carry over: a keyspace is also a consistency and topology boundary in Cassandra. In CockroachDB it is not — transactions span databases freely, and consistency is the same everywhere.

Quorum and Consistency

CockroachDB has no consistency levels. Every write commits at a **majority of the range's voting replicas**. What you control is *where* those voters sit, so the effect of `LOCAL_QUORUM` is achieved by placement rather than by a per-query flag:

CockroachDB setting	Voters	Quorum	Nearest Cassandra analogue
<code>SURVIVE ZONE FAILURE + REGIONAL BY ROW / BY TABLE</code>	3, all in the row's home region	2 of 3, inside the region	<code>LOCAL_QUORUM</code>
<code>SURVIVE REGION FAILURE</code>	5, across ≥ 3 regions	3 of 5, must include another region	<code>QUORUM</code> — every write pays one cross-region hop
Single-region cluster	3	2 of 3	<code>QUORUM</code> in a one-DC ring

Two differences to state plainly:

- It is decided **once, in the schema**, per database or table — never per statement by the application. There is no `ONE`, no `ALL`, and no way for a client to weaken a write.
- Non-voting replicas** exist but never count toward quorum. `GLOBAL` tables and follower reads serve reads locally from replicas outside the write quorum, so a read copy in every region does not widen the quorum.

The mental shift: consistency is fixed at "strong"; the knob you keep is **latency**, and you turn it by deciding how far apart the voters are.

Gossip

Same word, same mechanism, same Dynamo inheritance: nodes exchange what they know with a few peers and information spreads epidemically with no coordinator. Both use it for membership and node metadata.

The difference is what rides on it. In **Cassandra** gossip is central — membership, token ownership, schema version, and the failure detector all flow through it, so gossip trouble is a real outage class. In **CockroachDB**

it is peripheral: it carries node addresses, localities, store capacity, and cluster settings, all of which tolerate being seconds stale. Nothing correctness-critical depends on it:

- **Liveness** is a heartbeat into a replicated system range, not gossip; a lease is valid only while that heartbeat is current.
- **Range location** resolves through the replicated meta ranges; gossip only seeds the cache.
- **Schema** is a replicated table, not a gossiped version number.
- **Reads and writes** never touch gossip at all.

Operationally that means gossip lag in CockroachDB is cosmetic — the node list on the console briefly short by one after start-up — rather than a source of split-brain.

Where Correctness Lives

In Cassandra it lives in the **application**: you pick the consistency level, design idempotent writes, handle read-repair and tombstones, and denormalise for every query you will ever run.

In CockroachDB it lives in the **database**: the price is a consensus round on every write and a key-design discipline to keep writes spread — because sorted placement does not hash them for you.

Which to Choose

Cassandra when the workload is write-heavy with a known, narrow set of access patterns, row-level correctness under concurrency does not matter, and the lowest write latency at a consistency level *you* choose is the goal. Time-series ingest, activity feeds, IoT telemetry — anything modelled as "append here, read back by this key."

CockroachDB when you need the relational model — joins, constraints, ad-hoc queries — with transactions that are correct across rows, and survival across zones or regions without the application reasoning about consistency levels. Ledgers, inventory, accounts — anything where two writers touching the same data must not both succeed.

Three Surprises for a Cassandra Team

1. **Sequential keys hotspot.** Hashing was doing that job for you. Use UUIDs or hash-sharded keys (Lab 3).
2. **SQLSTATE 40001 retries are normal.** You never saw contention failures because you never had transactions. The application re-runs the transaction (Labs 5 and 14).
3. **Joins exist, and they are network operations.** Index the join column, use `STORING`, put small reference tables `GLOBAL`, keep related rows in the same region (Labs 4, 6, 7).

Term Map

Cassandra	CockroachDB
Keyspace	Database (plus zone configuration)
Table	Table
Partition key	Leading column(s) of the primary key
Clustering columns	Remaining primary key columns (sort order within the prefix)
Partition	Range (but dynamic, and ordered across ranges)
Token ring / vnodes	Sorted key space / range boundaries
Replication factor	<code>num_replicas</code> in the zone config
<code>NetworkTopologyStrategy</code>	Localities + survival goal + table locality
Consistency level	None — always a majority of voters
<code>LOCAL_QUORUM</code>	Zone survival with regional table locality
Coordinator node	Gateway node
Lightweight transaction	Any transaction
Materialized view / denormalised table	Secondary index (<code>STORING</code> for covering)
Tombstone	MVCC version, garbage-collected after <code>gc.ttlseconds</code>
<code>nodetool</code>	<code>cockroach node ...</code> , <code>crdb_internal</code> , DB Console
SSTable / compaction	SSTable / compaction (Pebble)